

Universidade de Vigo

Deseño e desenvolvemento dunha funcionalidade de
busca e filtrado de cursos no catálogo de cursos en liña

Aarón Riveiro Vilar

Traballo Fin de Grao
Escola de Enxeñaría de Telecomunicación
Grao en Enxeñaría de Tecnoloxías de Telecomunicación

Titores:
Manuel Caeiro Rodríguez

Curso 2025/2026

Índice

1. Introducción	1
1.1. Contexto do proxecto	1
1.2. Motivación	1
1.3. A plataforma de partida	2
2. Obxectivos e requisitos	2
2.1. Requisitos funcionais	3
2.1.1. Ordenación de resultados	3
2.1.2. Busca difusa	3
2.2. Requisitos non funcionais	3
3. Deseño	4
4. Desenvolvemento	4
5. Conclusións e liñas futuras	4
6. Referencias	4
A. Estado da arte	5
A.1. Busca difusa e correspondencia aproximada de cadeas	5
A.2. Busca e indexación en sistemas de xestión de contidos	5

1. Introducción

Nos últimos anos, o crecemento sostido dos sistemas de información dixitais deu lugar a repositorios con volumes de datos cada vez maiores e máis heteroxéneos. Neste contexto, a mera dispoñibilidade da información xa non resulta suficiente: é necesario dotar os sistemas de mecanismos eficaces que permitan localizar, filtrar e presentar os datos de forma relevante para o usuario. A disciplina da recuperación de información (*Information Retrieval*) estuda precisamente os métodos e modelos que permiten acceder de maneira eficiente a grandes coleccións de documentos, maximizando a relevancia dos resultados ofrecidos fronte a unha consulta determinada [1].

A medida que aumenta o tamaño e a complexidade dos repositorios, a experiencia de usuario pasa a depender en grande medida da calidade do sistema de busca e filtrado. Non só é importante a rapidez de resposta, senón tamén a precisión e exhaustividade dos resultados, así como a capacidade do sistema para interpretar a intención de busca do usuario e ofrecer mecanismos de refinamento progresivo mediante filtros, facetas ou criterios adicionais [2]. Un sistema de busca limitado a coincidencias literais sobre un único campo de texto pode resultar insuficiente en contornas nas que os datos presentan múltiples dimensións e atributos relevantes.

Por iso, o deseño e a implementación de estratexias avanzadas de busca e filtrado constitúen un elemento clave en calquera sistema de información orientado ao usuario final, especialmente cando se pretende facilitar o acceso eficiente a contidos estruturados e mellorar a usabilidade global da plataforma.

1.1. Contexto do proxecto

O proxecto DACEM (*Digitising Academic Catalogues for Enhanced Mobility*) [3], aprobado na convocatoria de 2023 do programa Erasmus+ para Asociacións Estratéxicas en Educación Superior (código de proxecto 2023-1-ES01-KA220-HED-000160344), ten como finalidade o desenvolvemento dunha plataforma de catálogo de cursos dirixida a institucións educativas europeas. O seu obxectivo principal é ofrecer un sistema que permita publicar e manter información actualizada sobre titulacións e materias de forma accesible, estruturada e interoperable, facilitando así os procesos de mobilidade académica internacional. A plataforma está orientada a dous perfís principais de usuarios: as institucións educativas, que a empregan para publicar e manter actualizado o seu catálogo de cursos; e os estudantes que, no marco de procesos de mobilidade Erasmus, consultan a oferta formativa das universidades parceiras.

O sistema de información asociado a esta plataforma está fundamentado en tecnoloxías de software libre e está concibido para poder despregarse tanto en contornas locais como en infraestruturas na nube. Máis alá do almacenamento estruturado de datos académicos, o correcto funcionamento do sistema depende da súa capacidade para xestionar, organizar e recuperar a información de maneira eficiente. Neste contexto, os mecanismos de busca e filtrado desempeñan un papel esencial, xa que constitúen o principal medio de interacción entre o usuario e o repositorio de cursos. A calidade destes mecanismos condiciona directamente a usabilidade do sistema, a precisión dos resultados obtidos e a experiencia global de navegación.

1.2. Motivación

Un dos retos fundamentais nos sistemas de información que xestionan grandes volumes de datos estruturados non reside unicamente no seu almacenamento, senón na forma en que estes datos son recuperados e presentados ao usuario. A medida que un repositorio medra en número de elementos e en complexidade de atributos, os mecanismos de busca simples baseados en coincidencias textuais directas poden resultar insuficientes para ofrecer resultados relevantes e axustados ás necesidades reais de consulta.

Esta problemática é especialmente significativa en contornas nas que os datos presentan múltiples dimensións descritivas —como titulacións, materias, áreas de coñecemento,

idiomas, créditos ou modalidades— e nas que o usuario pode ter criterios de busca diversos e combinados. A ausencia de mecanismos avanzados de filtrado e refinamento progresivo limita a capacidade do sistema para adaptarse a diferentes perfís de usuario e escenarios de consulta, reducindo a eficiencia na localización de información pertinente.

No ámbito actual do desenvolvemento de aplicacións web e sistemas de xestión de contidos, as tendencias apuntan cara á implementación de motores de busca máis sofisticados, capaces de indexar múltiples campos, ponderar resultados segundo a relevancia, incorporar criterios de filtrado dinámico e mellorar a experiencia de usuario mediante interfaces intuitivas. Estas solucións non só optimizan o acceso á información, senón que contribúen a unha maior usabilidade e aproveitamento do sistema no seu conxunto.

O presente Traballo de Fin de Grao enmárcase neste contexto, propoñendo o deseño e desenvolvemento dunha mellora nos mecanismos de busca e filtrado do catálogo de cursos da plataforma DACEM. O obxectivo é evolucionar dende un sistema de coincidencia textual básica cara a unha solución máis flexible e precisa, aliñada coas boas prácticas actuais en recuperación de información e desenvolvemento web, empregando tecnoloxías abertas e integradas na contorna existente. Concretamente, abórdase a implementación de criterios de ordenación de resultados para as vistas de busca existentes, así como un mecanismo de busca difusa que permita localizar resultados aínda en presenza de erros tipográficos na consulta.

1.3. A plataforma de partida

A plataforma DACEM está implementada sobre Drupal 10, un sistema de xestión de contidos (CMS) de código aberto amplamente empregado no ámbito académico e institucional. Drupal 10 proporciona unha contorna flexible para a definición de tipos de contido estruturado, a xestión de usuarios e a publicación de información a través de interfaces web configurables.

O modelo de datos organizase arredor de tres tipos de contido principais: *programme*, que representa unha titulación académica; *individual educational component* (IEC), correspondente a unha materia ou compoñente curricular individual; e *institution*, que modela as universidades participantes. Cada entidade conta cun conxunto de campos estruturados que recollen atributos como a área de coñecemento, o nivel de cualificación, os créditos, o idioma de impartición ou o país de orixe, entre outros.

Para a presentación e consulta do catálogo, a plataforma emprega o módulo *Views*, integrado no núcleo de Drupal, que permite definir listaxes e consultas de contido de forma declarativa. O módulo *Better Exposed Filters* complementa *Views* proporcionando a interface de filtrado ao usuario. Existen dúas vistas principais de busca: *search_programme*, accesible en `/search-programme`, orientada á consulta de titulacións; e *search_iec*, en `/search-iec`, orientada á consulta de compoñentes educativos individuais. A capa de presentación visual está baseada nun tema personalizado construído sobre Bootstrap 5.

O mecanismo de busca textual actual baséase no filtro *combine* de *Views*, que realiza comparacións de subcadeas sobre o campo título das entidades. Esta aproximación presenta limitacións relevantes: a busca restrínxese ao campo título, ignorando atributos como a descrición ou a área de coñecemento; non se aplica ningunha ponderación por relevancia, polo que os resultados se presentan en orde alfabética con independencia da pertinencia da consulta; e non existe tolerancia a erros tipográficos, de xeito que calquera inexactitude na consulta pode producir cero resultados aínda que o elemento buscado exista no catálogo. Ademais, as opcións de filtrado son estáticas e non se adaptan ao conxunto de resultados dispoñibles, polo que poden aparecer valores sen correspondencia con ningunha entrada existente.

2. Obxectivos e requisitos

O presente traballo ten como obxectivo principal mellorar a experiencia de busca e navegación do catálogo de cursos da plataforma DACEM. Partindo das limitacións identificadas

no sistema actual —ausencia de criterios de ordenación, busca por coincidencia exacta e sen tolerancia a erros tipográficos—, propónse o deseño e implementación de dúas mello-ras concretas: a incorporación de criterios de ordenación de resultados nas vistas de busca existentes, e a implementación dun mecanismo de busca difusa que permita localizar pro-gramas aínda en presenza de erros tipográficos na consulta.

A continuación defínense os requisitos funcionais e non funcionais que guían o desenvol-vemento. Enténdese por requisito funcional aquel que describe as capacidades concretas que debe ofrecer o sistema, mentres que os requisitos non funcionais establecen condi-cións relativas ao seu comportamento, integración e mantenibilidade, sen especificar fun-cionalidades concretas.

2.1. Requisitos funcionais

2.1.1. Ordenación de resultados

1. O sistema debe permitir ao usuario ordenar os resultados das vistas de busca segundo diferentes criterios, adaptados ao tipo de entidade que se consulta.
2. Cada criterio de ordenación debe permitir alternar entre orde ascendente e descen-dente mediante a mesma acción de usuario.
3. Ambas as vistas deben contemplar os seguintes criterios de ordenación comúns: orde alfabética por título, área de coñecemento, número de créditos e país de orixe da institución.
4. A vista de programas (*search_programme*) debe incluír adicionalmente o criterio de ordenación por nivel de cualificación EQF.
5. A vista de compoñentes educativos (*search_iec*) debe incluír adicionalmente os crite-rios de ordenación por programa asociado e cuadrimestre.
6. A interface de ordenación debe integrarse visualmente de forma coherente co deseño existente da plataforma.

2.1.2. Busca difusa

1. O sistema debe ser capaz de localizar resultados relevantes aínda cando a consulta introducida polo usuario conteña erros tipográficos ou grafías aproximadas.
2. A busca difusa debe aplicar un criterio de similaridade entre a consulta e os títulos dos elementos do catálogo, de maneira que se poidan recuperar resultados con pequenas desviacións ortográficas respecto ao texto almacenado.
3. A normalización da consulta debe incluír a conversión a minúsculas e a eliminación de diacríticos, de xeito que maiúsculas e caracteres acentuados non afecten aos resul-tados da busca.
4. Os resultados deben presentarse ordenados por relevancia, de xeito que os elementos con maior correspondencia coa consulta aparezan en primeiro lugar.
5. O mecanismo de busca difusa debe integrarse no fluxo de consulta existente, de xeito que o usuario non perciba diferenzas na interface respecto ao comportamento ante-rior.
6. O sistema debe manter un comportamento consistente ante consultas baleiras ou moi curtas, evitando erros ou resultados inesperados.

2.2. Requisitos non funcionais

1. A solución debe integrarse coa plataforma existente sen alterar a estrutura fundamen-tal do modelo de datos nin comprometer o funcionamento do resto de funcionalida-des do sistema.

2. O desenvolvemento debe estar fundamentado en tecnoloxías de código aberto, mantendo coherencia cos principios e a arquitectura do proxecto DACEM.
3. O sistema debe garantir un comportamento estable e previsible ante entradas incorrectas ou consultas mal formadas, evitando erros visibles para o usuario final.
4. A interface debe cumprir criterios básicos de usabilidade, asegurando que os mecanismos de interacción resulten intuitivos e coherentes co deseño xeral da plataforma.
5. O sistema debe garantir tempos de resposta axeitados nas consultas, de maneira que as melloras introducidas non degraden de forma perceptible a experiencia de uso.
6. A solución debe minimizar o impacto nos recursos do servidor, evitando sobrecargas innecesarias e optimizando as consultas realizadas á base de datos.
7. O rendemento do sistema debe manterse estable ante un incremento razoable do volume de datos almacenados no catálogo, asegurando que a solución escale de forma adecuada.
8. A implementación debe estruturarse de forma modular e documentada, de xeito que cada mellora poida ser mantida ou ampliada de forma independente sen afectar ao resto do sistema.

3. Deseño

4. Desenvolvemento

5. Conclusións e liñas futuras

6. Referencias

- [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] “Retrieval accuracy,” <https://www.sciencedirect.com/topics/computer-science/retrieval-accuracy>.
- [3] “Digitising academic catalogues for enhanced mobility,” <https://projects.uni-foundation.eu/dacem/>, European University Foundation, 07/2024.
- [4] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [5] F. J. Damerau, “A technique for computer detection and correction of spelling errors,” *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964.
- [6] W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage,” in *Proceedings of the Section on Survey Research Methods*. American Statistical Association, 1990, pp. 354–359.
- [7] T. Seidl *et al.*, “Search API,” https://www.drupal.org/project/search_api, 2024.

A. Estado da arte

A.1. Busca difusa e correspondencia aproximada de cadeas

A busca difusa (*fuzzy search*) é unha técnica de recuperación de información que permite localizar resultados relevantes aínda cando a consulta do usuario non coincide exactamente co texto almacenado. A diferenza da busca por subcadeas exactas, tolera desviacións ortográficas e erros tipográficos, o que mellora a experiencia de usuario en sistemas con datos introducidos manualmente ou con vocabulario técnico susceptible a equivocacións.

O fundamento teórico da busca difusa reside no concepto de distancia de edición entre dúas cadeas, definida como o número mínimo de operacións elementais necesarias para transformar unha cadea na outra. O algoritmo máis empregado é a **distancia de Levenshtein** [4], que considera tres operacións básicas: inserción, eliminación e substitución dun carácter. Dados dous textos de lonxitude m e n , calcúlase en tempo $O(m \times n)$ mediante programación dinámica.

A **distancia de Damerau-Levenshtein** [5] amplía este modelo incorporando as transposicións de caracteres adxacentes como operación atómica, o que a fai máis precisa para modelar erros tipográficos humanos, onde o intercambio de dúas letras consecutivas é un dos erros máis frecuentes. Outra métrica habitual é a **similitude de Jaro-Winkler** [6], que premia as cadeas con prefixo común e adoita empregarse na comparación de nomes propios. Por último, os enfoques baseados en **n-gramas** dividen as cadeas en secuencias solapadas de n caracteres e comparan a proporción de n-gramas compartidos, ofrecendo bo rendemento en textos longos e sendo facilmente paralelizables.

Para o presente traballo optouse pola distancia de Levenshtein como criterio de similaridade, pola súa sinxeleza conceptual, a súa robustez ante os tipos de erro tipográfico máis comúns e a súa idoneidade para cadeas curtas como os títulos de programas académicos.

A.2. Busca e indexación en sistemas de xestión de contidos

Os sistemas de xestión de contidos modernos incorporan habitualmente mecanismos de busca que van máis alá da simple consulta SQL. En Drupal 10, a solución integrada por defecto para a busca en listaxes de contido é o módulo *Views*, cuxo filtro *combine* realiza comparacións de subcadeas mediante operadores SQL LIKE. Esta aproximación límitase a coincidencias exactas e non ofrece capacidades de indexación nin de ponderación de resultados por relevancia.

Para cubrir estas limitacións, o ecosistema de Drupal conta co módulo *Search API* [7], que proporciona unha capa de abstracción para a indexación e consulta de contido. O módulo inclúe un sistema de procesadores configurables —como normalización de maiúsculas, transliteración de acentos, tokenización ou eliminación de palabras baleiras— que se aplican durante a indexación con independencia do *backend* escollido. Os *backends* dispoñibles máis empregados son os seguintes:

- **Backend de base de datos:** almacena o índice en táboas da base de datos relacional existente. Non require infraestrutura adicional e ofrece un rendemento axeitado para catálogos de tamaño moderado.
- **Apache Solr:** motor de busca de código aberto baseado en Lucene, optimizado para grandes volumes de datos e con capacidades avanzadas de ponderación de resultados por relevancia. Require un servidor Solr externo.
- **Elasticsearch:** solución distribuída orientada á alta dispoñibilidade e á escalabilidade horizontal, con capacidades similares a Solr. Tamén require infraestrutura adicional dedicada.

Para este traballo optouse polo *backend* de base de datos, dado que non require infraestrutura adicional, é suficiente para o volume de datos do catálogo DACEM, e permite integrar a lóxica de busca difusa directamente en PHP, empregando o índice para recuperar os títulos sobre os que aplicar a métrica de Levenshtein.